

Text-to-SQL Assistant Report

1. Project Overview

This project implements a local Text-to-SQL assistant for the Chinook Music Store SQLite database. The user asks a natural-language question in a Gradio interface. The system builds live schema context from SQLite metadata, sends the question and schema to a local Ollama model through LangChain, validates the generated SQL, executes only safe read-only queries, and returns the generated SQL, explanation, result table, and status.

The project uses:

- Gradio for the UI.
- LangChain LCEL for the Text-to-SQL pipeline.
- Ollama with llama3.1:8b as the local LLM.
- SQLite for the Chinook database.
- Pandas for query results.
- sqlparse plus custom validation for safety checks.

Conversation memory for follow-up questions is intentionally not implemented in this version.

2. Architecture Diagram

User question

- > Gradio Blocks UI
- > User input safety validation
- > SQLite schema extraction
- > LangChain ChatPromptTemplate
- > Ollama ChatOllama model
- > String output parser
- > JSON extraction
- > SQL safety validation
- > Schema reference validation
- > SQLite SELECT execution
- > Optional one-step SQL repair on failure
- > Pandas DataFrame
- > Gradio SQL / explanation / results / status outputs
- > Query log

3. Main Components

app.py

app.py defines the Gradio Blocks interface. It provides:

- A textbox for natural-language questions.
- A button and submit handler.
- A generated SQL textbox.
- An explanation textbox.
- A Gradio DataFrame for results.

- A status/error textbox.
- Example questions.

sql_engine.py

sql_engine.py contains the main Text-to-SQL engine. It:

- Builds the LangChain chain: ChatPromptTemplate -> ChatOllama -> StrOutputParser.
- Loads Ollama settings from environment variables.
- Parses model output as JSON.
- Validates unsafe user questions before calling the model.
- Validates generated SQL before execution.
- Executes safe SQL through the database layer.
- Logs each question, generated SQL, and status.
- Attempts one general SQL repair step when validation or execution fails.

prompt_templates.py

prompt_templates.py defines the main system and user prompt templates. The prompt includes:

- {schema} for live database schema context.
- {question} for the user question.
- Strict SELECT-only rules.
- JSON output requirements.
- Few-shot examples.
- Generic analytical SQL guidance for grouped totals, averages, rankings, and top-per-group queries.

The analytical guidance is intentionally general and not hardcoded to the Chinook schema.

db_setup.py

db_setup.py handles the SQLite database. It:

- Locates Chinook_Sqlite.sqlite.
- Attempts to download the database automatically if it is missing.
- Opens SQLite connections.
- Extracts table names, columns, and foreign-key relationships.
- Builds compact schema context for the LLM.
- Executes read-only SQL and returns a Pandas DataFrame.

safety.py

safety.py validates both user questions and generated SQL. It rejects:

- Empty SQL.
- SQL comments.
- Multiple statements.
- Destructive or non-read-only keywords.
- Statements that are not SELECT or WITH ... SELECT.

Blocked keywords include INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, REPLACE, PRAGMA, ATTACH, DETACH, and VACUUM.

schema_validator.py

schema_validator.py adds schema-aware validation. It checks qualified references such as Table.Column or alias.Column against real SQLite metadata and catches common LLM errors before execution, including:

- Referencing a table or alias not present in FROM or JOIN.
- Referencing a column that does not exist on the selected table.
- Using a qualified reference from the wrong table.

4. Evaluation Summary

The evaluation file is evaluation.csv.

It contains 30 test cases, exceeding the required minimum of 20. The columns are:

- Level
- Question
- Expected SQL
- Generated SQL
- Pass/Fail
- Notes

Current evaluation status:

Category	Count
Simple	5
Intermediate	7
Advanced	6
Edge case	2
Advanced Extra	6
Safety	4
Total	30

All 30 rows are currently marked PASS.

The evaluation covers:

- Simple counting and listing.
- Joins across artists, albums, tracks, customers, invoices, and employees.
- Aggregations and revenue calculations.
- Advanced grouped analytics.
- Edge cases returning zero rows.
- Safety prompts and SQL injection-style destructive requests.

5. Failure Case Analysis

Several difficult cases were identified during testing and then improved with general prompt and validation rules:

Aggregating at the wrong grain

For questions such as customers spending more than the average in their country, the model may compare individual invoice rows instead of first summing spending per customer. The fix is a general rule: aggregate detail rows at the requested entity grain before comparing

totals, averages, maximums, minimums, or ranks.

Missing requested metrics

For questions asking for both revenue and sold count, the model may return only one metric. The prompt now instructs the model to calculate and return every requested metric explicitly.

Top item per group

For questions such as the top revenue-generating artist in each country, the model may produce invalid alias or subquery references. The prompt and repair prompt now recommend a CTE with `ROW_NUMBER() OVER (PARTITION BY group ORDER BY metric DESC/ASC)` and filtering to row number 1.

Optional related data

For questions including support representatives or optional related entities, an inner join can hide rows with missing related records. The prompt now recommends `LEFT JOIN` and `COALESCE` where optional information may be missing.

Schema hallucination

The model can still propose table or column names that do not exist. This is mitigated by live schema context, strict prompt rules, and `schema_validator.py`.

6. Technical Challenges

- Local LLM output can be inconsistent, so the engine must parse and validate model output defensively.
- Complex analytical questions require the model to choose the correct result grain before writing SQL.
- SQLite date and ranking logic requires specific syntax such as `strftime` and window functions.
- SQL safety must be enforced outside the LLM because prompt instructions alone are not enough.
- Schema validation is useful but remains a practical checker, not a complete SQL parser.
- The app depends on a local Ollama server and the configured model being available.

7. Improvements Made

- Added user-input validation before model invocation.
- Added SQL safety validation before database execution.
- Added schema-aware validation for table and column references.
- Added a general SQL repair step.
- Strengthened prompt rules for grouped analytics, averages, rankings, requested metrics, alias scope, and optional joins.
- Added automatic database download when `Chinook_Sqlite.sqlite` is missing.
- Updated README setup instructions.
- Added a PDF report artifact.

8. Current Limitations

- No conversation memory or follow-up question history.
- The generated SQL can still be verbose or include unnecessary joins.
- The repair step improves reliability but cannot guarantee correctness for every query.
- The schema validator does not fully parse every possible SQL construct.
- The system currently targets SQLite syntax.
- Result visualizations and CSV export are not implemented.

9. Future Improvements

- Add conversation memory for follow-up questions.
- Add an automated evaluation runner that executes every row in evaluation.csv.
- Add charts for revenue, ranking, and time-series questions.
- Add downloadable result exports.
- Add deeper SQL parsing for validation.
- Add support for other SQL dialects and databases.

10. Conclusion

The project satisfies the main requirements for a local Text-to-SQL assistant: it accepts natural-language questions, generates SQL, displays SQL before execution, executes only safe read-only queries, shows results in Gradio, handles invalid and unsafe inputs gracefully, logs executions, includes a populated evaluation sheet, and provides written documentation and a PDF report.